

## COMP 9057 DECISION ANALYTICS



### ASSIGNMENT 1 – Constraint Programming

#### SUBMISSION & DUE DATE

This assignment should be submitted to Canvas before 11:59pm on **Friday 6/11/2026**. The usual late submission penalties apply in accordance with MTU's marks and standards policy.

Please submit a single ZIP file with your student number and name in the filename. Your submission should contain **exactly 2 files**:

- A detailed .pdf documentation of your code including all outputs and results that generated. Please make sure that you are referencing the relevant tasks and explain your reasoning.
- All Python code in a single .py file that can be executed and that generates the outputs you are referring to in your evaluation. The file needs to be readable in a plain text editor. Please do NOT submit a notebook file or repository link. Make sure that you clearly indicate in your comments the exact subtask every piece of code is referring to.

Please do **NOT** include any other files (input files, OS or IDE metadata, etc.) in your submission. Your code should load the input files from the current directory only and must not create any output files or directories, so that it can run anywhere without modification.

You can achieve a total of 50 points for the submission as indicated in the tasks. For each subtask you are given full marks for correct answers, 70% for minor mistakes, and 35%, 15%, or 0% for major mistakes or omissions depending on severity.

#### TASK A (logistics problem, 30 points)

In this task you will develop a constrained programming model and use the OR-Tools CP-SAT solver to find a solution for a logistics and distribution scenario. The input data is available on Canvas in the Excel file *Assignment\_DA\_1\_Logistics.xlsx*.

In this scenario, you are optimising four warehouses each of which is operating several trucks bringing in products from suppliers to be processed, repackaged, and then shipped to customers using smaller delivery vans. The number of supply trucks and delivery vans available at each warehouse is provided in the *warehouses* table of the input file.

The customers are grouped into delivery locations and for each location the *customer orders* table indicates the quantity for each product (A-Z) that must be delivered there. Similar, the *supplier stock* table shows for each location how much of each product is available for collection there. Note, that everything must be processed through the warehouses, so even if a product is available in the same location as a customer order, it must be collected by a supply truck first, transported back to the warehouse, before it can be delivered to the customer in a delivery van.

The capacity of supply trucks and delivery vans is limited. Supply trucks can carry a maximum of 15,000kg and delivery vans are limited to carrying no more than 3,000kg of product. The weight of each product type (in kg) is provided in the *product spec* table.

The goal is to find optimal tours for each of the supply trucks as well as the delivery vans. To facilitate this, the *distances* table contains the mutual distances between all locations.

1. Identify the decision variables for the problem, then create a CP-SAT model and add the necessary variables to the model. [5 points]
2. Identify the constraints of the problem and add them to the model. Clearly describe the purpose of each of your constraints and reference it in your code. [10 points]
3. Optimising the total distance travelled by each vehicle is likely infeasible. As an approximation, introduce an auxiliary variable for each vehicle containing its radius, i.e. the distance between the warehouse and the furthest location the vehicle is serving. Then use the solver to minimise the maximum of all these radii. [5 points]
4. Using the optimal solution from the previous task, build CP-SAT models for each of the vehicles individually to determine optimal tours that minimise the total distance travelled while visiting all locations they serve. [5 points]
5. Output for each product how much of it is processed in each warehouse, on which trucks it is arriving, and on which vans it is departing. Also output for each of the vehicles, the route it is travelling and what quantities of which product it is collecting from or delivering to each of the locations on its route. [5 points]

### TASK B (sudoku solver, 20 points)

In this task you will develop a constraint programming solver to find the solution for the following sudoku puzzle. Do not use the OR Tools library but instead implement the solver from scratch.

|   |   |   |   |   |   |  |   |   |
|---|---|---|---|---|---|--|---|---|
| 4 | 8 |   |   |   |   |  |   | 5 |
|   |   | 2 |   |   | 8 |  | 4 | 6 |
|   |   | 6 |   | 9 | 5 |  |   |   |
|   |   | 3 |   | 2 | 4 |  | 8 | 9 |
| 6 | 2 | 4 |   |   |   |  | 5 |   |
| 8 |   | 5 | 6 | 7 | 1 |  |   |   |
|   |   | 1 |   |   |   |  |   |   |
| 5 |   |   |   |   |   |  |   |   |
|   |   | 9 |   | 6 |   |  | 2 |   |

1. Create a data structure that can represent the necessary decision variables and their respective domains for a sudoku puzzle and initialise it with the values above. [5 points]
2. Implement an algorithm to efficiently propagate the *all-different* constraint over a subset of these decision variables. Make sure that the propagator you implement is monotonic and idempotent and provide an argument why this is the case. [5 points]
3. Implement a function that uses the *all-different* propagator to propagate the constraints of the sudoku puzzle, i.e. that no digit occurs twice in any of the rows, in any of the columns, or in any of the 3x3 sub-grids. [5 points]
4. Implement a backtracking search algorithm that alternates between propagation and branching. Choose an effective branching strategy and provide an argument why this strategy is most suitable for your propagator implementation. Output the solution for the sudoku problem above. [5 points]